📘 자바스크립트 최신 문법 (ES6 \~ ES11)

강의 소개

- 자바스크립트 개발자가 반드시 알아야 할 최신 문법 핵심 정리.
- **ES6(2015)**부터 **ES11(2020)**까지 자주 사용되는 문법을 다룸.
- 리액트(React) 및 다양한 라이브러리 개발 시 필수.
- 구형 브라우저 지원 문제로 **Babel**을 이용하는 것을 권장.
- 기초가 부족한 경우, 기본편과 보충설명 강의를 먼저 참고.

ES6 핵심 문법

1. Shorthand Property Names

- 객체를 생성할 때 key와 value의 이름이 동일하면 축약 가능.
- 코드가 간결해지고 가독성이 향상됨.
- **활용 예시**: API 요청 결과를 객체에 담을 때 변수명을 그대로 활용.

2. Destructuring Assignment

- 객체/배열의 값을 쉽게 분해하여 변수에 할당 가능.
- 새로운 변수명으로 매핑도 가능.
- **활용 예시**: React의 `useState` 훅 사용 시 `[state, setState]` 구조 분해.

3. Spread Syntax

- 배열/객체를 복사하거나 병합할 때 활용.
- 얕은 복사(shallow copy)만 수행 → 참조값 주의.
- 배열 합치기, 객체 병합, 새로운 요소 추가에 유용.
- **활용 예시**: Redux reducer에서 불변성 유지 시 사용.

4. Default Parameters

- 함수 정의 시 매개변수에 기본값을 지정 가능.
- 인자가 전달되지 않으면 기본값 사용.
- **활용 예시**: API 함수에서 `page=1` 같은 기본값 처리.

5. Ternary Operator

- 조건문을 간단히 표현.
- `조건 ? 값1 : 값2` 형태.
- 리액트에서 조건부 렌더링 시 자주 사용.
- **활용 예시**: 사용자 로그인 여부에 따른 UI 분기.

6. Template Literals

- 백틱(``)을 이용하여 문자열 보간 가능.

- 가독성이 뛰어나며 줄바꿈도 쉽게 처리 가능.
- **활용 예시**: 다국어(i18n) 문자열 조합.

ES11 핵심 문법

1. Optional Chaining

- 객체 속성이 존재하지 않아도 안전하게 접근 가능.
- 중첩된 객체 접근 시 에러 방지.
- **활용 예시**: API 응답 데이터의 nested 값 접근.

2. Nullish Coalescing Operator

- `null` 또는 `undefined` 일 경우에만 대체값을 할당.
- OR 연산자(`||`)와 달리 falsy 값(`0` , `''`)은 그대로 유지.
- **활용 예시**: 사용자 입력값이 비어 있을 때 기본값 설정.

📑개발 코드 자료

```
// 개발 코드 자료: es11.js
/**
 * Optional Chaining (ES11)
 * https://developer.mozilla.org/ko/docs/Web/JavaScript/Reference/Operators/Optional_chaining
 */
{
  const person1 = {
    name: 'Ellie',
    job: {
      title: 'S/W Engineer',
      manager: {
        name: 'Bob',
      },
    },
  };
  const person2 = {
    name: 'Bob',
  };

  // 💩💩💩💩💩💩
  {
    function printManager(person) {
      console.log(person.job.manager.name);
    }
    printManager(person1);
    // printManager(person2);
  }
```

```
  // 💩💩💩
  {
    function printManager(person) {
      console.log(
        person.job
          ? person.job.manager
            ? person.job.manager.name
            : undefined
          : undefined
      );
    }
    printManager(person1);
    printManager(person2);
  }

  // 💩
  {
    function printManager(person) {
      console.log(person.job && person.job.manager && person.job.manager.name);
    }
    printManager(person1);
    printManager(person2);
  }

  // 👎
  {
    function printManager(person) {
      console.log(person.job?.manager?.name);
    }
    printManager(person1);
    printManager(person2);
  }
  console.clear();
}

/**
 * Nullish Coalescing Operator (ES11)
 * https://developer.mozilla.org/ko/docs/Web/JavaScript/Reference/Operators/
Nullish_coalescing_operator
 */
{
  // Logical OR operator
  // false: false, '', 0, null, undefined
  {
    const name = 'Ellie';
    const userName = name || 'Guest';
    console.log(userName);
  }

  {
    const name = null;
```

```
  const userName = name || 'Guest';
  console.log(userName);
 }

 // 💩
 {
  const name = '';
  const userName = name || 'Guest';
  console.log(userName);

  const num = 0;
  const message = num || 'undefined';
  console.log(message);
 }

 // 👎
 {
  const name = '';
  const userName = name ?? 'Guest';
  console.log(userName);

  const num = 0;
  const message = num ?? 'undefined';
  console.log(message);
 }
}
```

```
// 개발 코드 자료: es6.js
/**
 * Shorthand property names
 * https://developer.mozilla.org/ko/docs/Web/JavaScript/Reference/Operators/Object_initializer
 *
 */
{
 const ellie1 = {
  name: 'Ellie',
  age: '18',
 };

 const name = 'Ellie';
 const age = '18';

 // 💩
 const ellie2 = {
  name: name,
  age: age,
 };

 // 👎
 const ellie3 = {
```

```
  name,
  age,
 };

 console.log(ellie1, ellie2, ellie3);
 console.clear();
}

/**
 * Destructuring Assignment
 * https://developer.mozilla.org/ko/docs/Web/JavaScript/Reference/Operators/
Destructuring_assignment
 *
 */
{
 // object
 const student = {
  name: 'Anna',
  level: 1,
 };

 // 💩
 {
  const name = student.name;
  const level = student.level;
  console.log(name, level);
 }

 // 👎
 {
  const { name, level } = student;
  console.log(name, level);

  const { name: studentName, level: studentLevel } = student;
  console.log(studentName, studentLevel);
 }

 // array
 const animals = ['🖤', '😼'];

 // 💩
 {
  const first = animals[0];
  const second = animals[1];
  console.log(first, second);
 }

 // 👎
 {
  const [first, second] = animals;
  console.log(first, second);
```

```
  }
  console.clear();
}

/**
 * Spread Syntax
 * https://developer.mozilla.org/ko/docs/Web/JavaScript/Reference/Operators/Spread_syntax
 *
 */
{
  const obj1 = { key: 'key1' };
  const obj2 = { key: 'key2' };
  const array = [obj1, obj2];

  // array copy
  const arrayCopy = [...array];
  console.log(array, arrayCopy);

  const arrayCopy2 = [...array, { key: 'key3' }];
  obj1.key = 'newKey';
  console.log(array, arrayCopy, arrayCopy2);

  // object copy
  const obj3 = { ...obj1 };
  console.log(obj3);

  // array concatenation
  const fruits1 = ['💖', '💙'];
  const fruits2 = ['👺', '😳'];
  const fruits = [...fruits1, ...fruits2];
  console.log(fruits);

  // object merge
  const dog1 = { dog: '🤪' };
  const dog2 = { dog: '🖤' };
  const dog = { ...dog1, ...dog2 };
  console.log(dog);
  console.clear();
}

/**
 * Default parameters
 * https://developer.mozilla.org/ko/docs/Web/JavaScript/Reference/Functions/Default_parameters
 */
{
  // 💩
  {
    function printMessage(message) {
      if (message == null) {
        message = 'default message';
      }
```

```
    console.log(message);
  }

  printMessage('hello');
  printMessage();
 }

 // 👎
 {
  function printMessage(message = 'default message') {
    console.log(message);
  }

  printMessage('hello');
  printMessage();
 }
 console.clear();
}

/**
 * Ternary Operator
 * https://developer.mozilla.org/ko/docs/Web/JavaScript/Reference/Operators/
Conditional_Operator
 */
{
 const isCat = true;
 // 💩
 {
  let component;
  if (isCat) {
    component = '😸';
  } else {
    component = '🖤';
  }
  console.log(component);
 }

 // 👎
 {
  const component = isCat ? '😸' : '🖤';
  console.log(component);
  console.log(isCat ? '😸' : '🖤');
 }
 console.clear();
}

/**
 * Template Literals
 * https://developer.mozilla.org/ko/docs/Web/JavaScript/Reference/Template_literals
 */
{
```

```
  const weather = '😢';
  const temparature = '16°C';

  // 💩
  console.log(
    'Today weather is ' + weather + ' and temparature is ' + temparature + '.'
  );

  // 👎

  console.log(`Today weather is ${weather} and temparature is ${temparature}.`);

}
```

📖요약 & 학습 포인트

- **ES6**: 객체 축약, 구조 분해 할당, 스프레드 문법, 기본 매개변수, 삼항 연산자, 템플릿 리터럴.
- **ES11**: 옵셔널 체이닝, 널 병합 연산자.
- 코드 작성 시 가독성과 유지보수를 높이는 최신 문법 적극 활용.